Architecture des Ordinateurs : Représentation des Entiers et Arithmétique en Complément à Deux

Thème principal : Représentation matérielle des entiers signés, conversion en complément à deux, arithmétique binaire et détection du dépassement de capacité (Overflow)

Synthèse exécutive : Ce cours aborde les mécanismes fondamentaux utilisés par les ordinateurs pour coder et manipuler les nombres entiers relatifs (positifs et négatifs) en mémoire binaire. En raison de l'impossibilité matérielle pour les circuits électroniques (bascules et transistors) de stocker un symbole abstrait '+' ou '-', l'architecture informatique moderne repose sur la convention du complément à deux sur une largeur fixe de N bits. Ce système résout les faiblesses historiques du codage Signe et Magnitude en garantissant l'unicité de la représentation du zéro (suppression du double zéro) et en unifiant l'addition et la soustraction au sein d'un circuit additionneur unique dans l'Unité Arithmétique et Logique (ALU). Le cours détaille la méthode de conversion en trois étapes, le calcul de la plage de valeurs dissymétrique [-2^(N-1), +2^(N-1)-1], la gestion de la retenue sortante (Carry Out), ainsi que la règle formelle de détection du dépassement de capacité (Overflow), source classique de bogues critiques et de perte de points aux examens.

1. Problématique matérielle et justification du Complément à Deux

Au niveau le plus bas de l'architecture matérielle d'un processeur, la mémoire vive et les registres sont composés de circuits électroniques bistables (bascules). Ces composants ne reconnaissent que deux niveaux de tension électrique : 0 Volt (0 logique) et la tension d'alimentation Vcc (1 logique). Pour représenter les nombres négatifs, plusieurs conventions ont été historiquement envisagées. La plus intuitive, le codage 'Signe et Magnitude', s'est avérée inefficace. Elle génère deux représentations distinctes pour zéro (+0 et -0), gaspillant une possibilité de codage et imposant au processeur de tester deux motifs binaires différents pour vérifier si une valeur est nulle. De plus, elle contraint les concepteurs de puces à intégrer deux circuits matériels séparés dans l'Unité Arithmétique et Logique (ALU) : un pour l'addition et un pour la soustraction. La convention du complément à deux résout intégralement ces contraintes. D'une part, le nombre zéro possède une représentation unique composée exclusivement de bits à 0. D'autre part, la soustraction A - B est mathématiquement ramenée à l'addition A + (-B). Cela signifie que le même additionneur binaire matériel est capable d'effectuer à la fois des additions et des soustractions, simplifiant considérablement l'architecture interne des microprocesseurs.

- • Absence de signe physique : les bascules électroniques et les transistors des registres processeur ne stockent que deux états logiques (0 pour 0 Volt, 1 pour Vcc). Aucun composant physique ne peut enregistrer directement le symbole '+' ou '-'.
- • Limites du codage 'Signe et Magnitude' : cette méthode réserve le bit le plus à gauche pour le signe (0 = positif, 1 = négatif) et le reste pour la valeur absolue. Elle produit deux codages distincts pour zéro (+0 = 0000_2 et -0 = 1000_2 sur 4 bits), ce qui gaspille une combinaison binaire, alourdit la logique de comparaison d'égalité et exige deux circuits physiques distincts dans l'ALU (un additionneur et un soustracteur).
- • Unification par le Complément à Deux : cette convention donne au zéro un codage unique (0000...0) et transforme toute soustraction A - B en une addition A + (-B), permettant de réutiliser le même circuit additionneur au niveau de l'ALU.

Cas concret enseignant : *La métaphore du compteur kilométrique d'une voiture ancienne à 4 chiffres : si le compteur affiche 0000 km et que vous effectuez un kilomètre en marche arrière, le compteur repasse mécaniquement à 9999. En binaire sur 8 bits, faire 00000000_2 - 1 donne 11111111_2, ce qui définit formellement la représentation binaire de -1 en complément à deux.*

2. Algorithme de Conversion et Plage de Valeurs sur N bits

Pour convertir un nombre négatif -X en complément à deux, il faut obligatoirement définir au préalable la taille N du mot mémoire (ex: 8, 16, 32 ou 64 bits). L'algorithme de conversion s'exécute en trois étapes séquentielles strictes : 1. Conversion de la valeur absolue X en binaire naturel, complétée impérativement par des 0 à gauche jusqu'à

atteindre la taille exacte de N bits (cadrage obligatoire). 2. Application du complément à un : chaque bit de la séquence est inversé (les 0 deviennent 1, les 1 deviennent 0). 3. Application du complément à deux : on ajoute la valeur 1 au mot binaire obtenu à l'étape 2. Le bit de poids fort (MSB - Most Significant Bit), situé tout à gauche, prend la valeur 1 pour tout entier négatif et la valeur 0 pour un entier positif ou nul. En raison du rattachement de la combinaison zéro (0000...0) au domaine positif (MSB = 0), la plage des valeurs représentables sur N bits est dissymétrique : elle comporte exactement un nombre négatif de plus que de nombres positifs. Sur un octet (8 bits), la plage s'étend ainsi de -128 à +127.

- Méthode des 3 étapes : 1. Écrire la valeur absolue X en binaire pur sur N bits complets ; 2. Inverser chaque bit (Complément à 1) ; 3. Ajouter 1 au résultat binaire (Complément à 2).
- Rôle du MSB (Most Significant Bit) : le bit le plus à gauche indique le signe (0 = positif ou nul, 1 = négatif).
- Dissymétrie de la plage de valeurs : sur N bits, la plage s'étend de -2^(N-1) à +(2^(N-1) - 1). Sur 8 bits, elle va de -128 à +127.

Cas concret enseignant : *Avertissement crucial pour les contrôles : il est IMPÉRATIF de fixer la taille N bits (ex: 8 bits) et d'écrire les zéros de gauche AVANT d'effectuer l'inversion des bits. Si l'on inverse le mot '1100' (+12 sur 4 bits) sans ajouter au préalable les 4 zéros de cadrage '00001100', on obtient '0011' (+3) au lieu de '11110011' (-12 sur 8 bits), ce qui conduit à un résultat faux en examen !*

3. Arithmétique Binaire, Retenue (Carry) et Dépassement de Capacité (Overflow)

En travaillant sur des registres de largeur fixe (8, 16, 32 ou 64 bits), les opérations arithmétiques peuvent générer deux drapeaux (flags) distincts au niveau du registre d'état du processeur : le Carry Out (C) et l'Overflow (OV). En arithmétique signée en complément à deux, le Carry Out représente la retenue sortante du bit MSB. Cette retenue est ignorée dans le calcul car le processeur fonctionne selon une arithmétique modulaire de période 2^N. Il ne faut pas la confondre avec l'Overflow. L'Overflow signale une corruption du bit de signe due à une capacité insuffisante du registre. La règle formelle de détection d'un Overflow lors d'une addition en contrôle est stricte : un Overflow se produit si et seulement si deux opérandes ayant le même bit de signe (deux positifs ou deux négatifs) sont additionnés et fournissent un résultat avec un bit de signe opposé. Si l'on additionne un nombre positif et un nombre négatif, l'apparition d'un Overflow est physiquement et mathématiquement impossible.

- Soustraction unifiée : calculée directement par l'ALU sous la forme A - B = A + (-B).
- Gestion de la retenue sortante (Carry Out) : la retenue générée au-delà du bit de poids fort (position N+1) est systématiquement ignorée par le processeur car l'arithmétique signée s'effectue modulo 2^N.
- Drapeau d'Overflow (OV) : indique un dépassement de capacité arithmétique où le résultat réel de l'opération excède la plage autorisée [-2^(N-1), +2^(N-1)-1].
- Règle d'or de l'Overflow : un dépassement de capacité survient si et seulement si l'addition de deux opérandes de même signe (deux positifs ou deux négatifs) produit un résultat de signe opposé. L'addition de deux nombres de signes différents ne peut jamais générer d'Overflow.

Cas concret enseignant : *L'anecdote historique du crash d'Ariane 5 (vol 501 en 1996) : une conversion de type d'un nombre flottant sur 64 bits vers un entier signé sur 16 bits sans protection contre le dépassement de capacité a provoqué un Overflow non capturé. L'Unité de Référence Inertielle a transmis une valeur corrompue au calculateur de bord, interprétée à tort comme une déviation de trajectoire, ce qui a déclenché le braquage brutal des tuyères puis l'autodestruction de la fusée.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la retenue tout à gauche (Carry sortant) est-elle jetée lors de l'addition en complément à deux ?
Réponse du professeur : La retenue sortante au-delà du N-ième bit est rejetée parce que les registres du processeur ont une taille fixe et fonctionnent en arithmétique modulaire modulo 2^N. Mathématiquement, l'opération du complément à deux consiste à représenter un nombre négatif -X par la valeur (2^N - X). Lors de l'addition A + (2^N - B) = 2^N + (A - B), la valeur 2^N ajoutée artificiellement crée cette retenue de poids 2^N à la position N+1. L'éliminer physiquement revient à retrancher ce 2^N pour obtenir le résultat exact (A - B).

Question étudiant : Comment l'ordinateur fait-il la différence entre le mot binaire 11111111 qui vaut 255 en non-signé et -1 en complément à deux ?
Réponse du professeur : Physiquement au niveau des transistors, l'ordinateur ne fait aucune différence : ce sont exactement les mêmes 8 bits stockés en mémoire. La différence réside dans l'instruction d'assemblage exécutée par le processeur (ou le type de donnée déclaré par le programmeur en C, comme 'unsigned char' ou 'signed char'). L'ALU utilisera des instructions d'évaluation des drapeaux différentes (ex: test du flag Carry pour le non-signé, test du flag Overflow pour le signé) selon le contexte de l'instruction.

■ Exercice & Démonstration d'Application

Énoncé : Sur une architecture 8 bits, effectuer la soustraction signée : 35 - 75. Détailler l'écriture binaire des deux opérandes en complément à deux, poser l'addition binaire correspondante, donner le résultat décimal et indiquer la valeur du drapeau d'Overflow (OV).

Résolution pas à pas :
Étape 1 : Conversion du premier opérande (+35) sur 8 bits. 35 = 32 + 2 + 1 = 00100011_2. Étape 2 : Conversion du second opérande (-75) en complément à deux sur 8 bits. - Valeur absolue (+75) en binaire naturel sur 8 bits : 64 + 8 + 2 + 1 = 01001011_2. - Inversion de tous les bits (Complément à 1) : 10110100_2. - Ajout de 1 (Complément à 2) : 10110100 + 1 = 10110101_2 (Représentation de -75). Étape 3 : Pose et exécution de l'addition binaire (+35) + (-75). 00100011 (+35) + 10110101 (-75) ---------- 11011000_2 Étape 4 : Interprétation du résultat binaire 11011000_2. Le bit le plus à gauche (MSB) vaut 1, le nombre est négatif. Pour trouver sa valeur absolue en décimal, on réapplique l'algorithme du complément à deux sur 11011000_2 : - Inversion des bits : 00100111_2 - Ajout de 1 : 00100111 + 1 = 00101000_2 = 32 + 8 = 40 décimal. Le mot binaire 11011000_2 représente donc la valeur -40 en décimal. Étape 5 : Analyse du drapeau de dépassement de capacité (Overflow). On additionne un nombre positif (+35, MSB=0) et un nombre négatif (-75, MSB=1). Les deux opérandes possédant des signes opposés, la production d'un Overflow est impossible.

Résultat : Résultat binaire : 11011000_2 (-40 en décimal). Drapeau d'Overflow OV = 0 (aucun dépassement de capacité). Le calcul 35 - 75 = -40 est exact et valide.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la plage exacte des entiers relatifs représentables sur un mot mémoire de 16 bits en complément à deux ?

R : La plage s'étend de -2^(16-1) à +(2^(16-1) - 1), c'est-à-dire de -32 768 à +32 767.

Q : Comment détecte-t-on de façon certaine la présence d'un Overflow lors d'une addition de deux entiers signés ?

R : Un Overflow survient si et seulement si l'addition de deux opérandes de même signe (deux positifs ou deux négatifs) produit un résultat de signe opposé.

Q : Quel est le résultat de l'application de la procédure du complément à deux sur la valeur zéro (00000000_2) sur 8 bits ?

R : Inversion des bits : 11111111_2. Ajout de 1 : 11111111 + 1 = 100000000_2. En conservant 8 bits, le résultat est 00000000_2 avec une retenue sortante (Carry Out) rejetée. Cela prouve l'unicité du zéro en complément à deux.